

3.iii. Estructuras de control

■ **Estructura condicional simple: *if***

**if (expresión) sentencia;
else sentencia;**

- La sentencia puede ser simple o puede ser un bloque de sentencias entre llaves { }.
- La expresión es verdadera si su valor es distinto de cero.
- El *else* es opcional.
- Se corresponde con la estructura SI-ENTONCES-SI-NO.

```
if(a>10)
    a++;
else {
    a--;
    c=a+b;
};
```

■ ***If* anidados**

**if(expresión1) sentencia 1;
else
 if(expresión2) sentencia 2;
 ...
else
 sentencia N;**

- C permite 15 niveles de anidamiento.
- El *else* va con el *if* más próximo. Para romper esta relación se utilizan las llaves { }.

```
if(a+b>10)
    a++;
else
    if(a<5)
        if(b<5)
            b=a*5;
        else
            c=a*5;
    else
        c=a+b;
```

3.iii. Estructuras de control

■ Estructura de selección múltiple: *switch*

```
switch(expresión) {  
    case constante 1:  
        secuencia de sentencias 1;  
    case constante 2:  
        secuencia de sentencias 2;  
    ...  
    default:  
        secuencia de sentencias n;  
};
```

- **default:** Evalúa la secuencia de sentencias cuando ningún valor de las constantes es igual a la expresión.
- **break:** Indica que no se continua con las demás sentencias de los *cases*. Cuando una constante es igual a la expresión, se ejecutan todas las sentencias hasta el final.
- **switch:** Evalúa sólo una igualdad, mientras que *if* puede evaluar varias expresiones.

```
switch(opc)  
{  
    case 'a':  
        a++;  
        break;  
    case 'b':  
        b--;  
        break;  
    default:  
        a=b;  
};
```

```
switch(x) {  
    case 1:  
    case 2:  
    case 3:  
        i=0;  
        break;  
    case 4:  
        i=1;  
    case 5:  
        switch(y) {  
            case 0:  
  
                printf("cero");  
                break;  
            case 1:  
  
                print("uno");  
            };  
        default:  
            procesar(c);  
    };  
};
```

3.iii. Estructuras de control

■ **Bucle por contador: *for***

**for(inicialización; condición; incremento)
sentencia;**

- **inicialización**: Expresión de inicialización o asignación de la variable o variables de control.
- **condición**: Expresión relacional que indica cuándo acaba el bucle.
- **incremento**: Informa de cómo varía la variable de control.
- **sentencia**: Puede ser una sentencia, un bloque de sentencias o incluso una sentencia vacía.
- El bucle **se repite mientras la condición sea verdadera**, y ésta se comprueba al principio del bloque.
- Se puede **utilizar el operador coma (,)** para incluir **dos o más inicializaciones**, condiciones o incrementos.
- La **inicialización, condición e incremento son opcionales**.
- Para que **finalice un for con dos o más condiciones se deben verificar todas**.
- **Bucle infinito**: **for(;;) sentencia;**

```
#include <stdio.h>
main()
{
    int x;
    for(x=1; x<=100; x++)
        printf("%d",x);
}
```

```
main()
{
    int i,j=0;

    for(i=10;i>0;i--)
    {
        j=j+1;
        printf("%d, ",j);
    }
    r[j]='\0';
}
```

3.iii. Estructuras de control

■ **Bucle por condición: *while***

```
while (condición)
{
    sentencia 1;
    . . . .
    sentencia N;
}
```

- **condición**: Expresión relacional que indica cuándo acaba el bucle.
- **sentencia**: Puede ser una sentencia, un bloque de sentencias o incluso una sentencia vacía.
- El bucle **se repite mientras la condición sea verdadera**. La condición se comprueba al principio del bloque.

```
char c='b';
```

```
while(c != 'A')
    c=getchar();
```

```
printf("%c", c);
```

```
int long, c, l=3;
```

```
long=19;
```

```
while(l<long) {
    c=c+2;
    l++;
};
```

```
printf("(%d,%d,%d)",l,long,c);
```

3.iii. Estructuras de control

■ **Bucle por condición: *while* y *do/while***

```
do
{
    sentencia 1;
    sentencia 2;
}
while(condición);
```

```
int num;

do {
    scanf("%d",&num);
} while(num>100);

printf("%d",num);
```

- **condición:** Expresión relacional que indica cuándo acaba el bucle.
- **sentencia:** Puede ser una sentencia, un bloque de sentencias o incluso una sentencia vacía.
- Primero se realiza la iteración y luego se evalúa la condición.
- Esto se repite mientras se cumple la condición.

8.2. Programación estructurada. Estructuras de control (V)

ESTRUCTURAS DE SALTO

■ **break**

- Se utiliza para salir de un bucle, continuando la ejecución en la siguiente sentencia al bucle.
- Cuando hay bucles anidados se sale del más interno.
- Sintaxis: **break N;**

■ **continue**

- Fuerza una nueva iteración del bucle ignorando el código del bucle.
- Sintaxis: **continue;**

■ **return**

- Se utiliza para salir de una función y volver al punto donde se invocó o llamó a ésta.
- En una función puede haber tantos *return* como se crea necesario.
- Sintaxis: **return N;**

```
main()
{
    int x;
    for(x=0; x<=100; x++)
        if (i!=30) printf("%d",x);
        else break;
}
```

```
main()
{
    int x;
    for(x=0; x<=100; x++)
        if (i!=30) printf("%d",x);
        else continue;
}
```